

- ✓ Aim: Develop a game-playing agent using minimax and alpha-beta pruning for Tic-Tac-Toe or a similar turn-based game.

```
board=[" "]*9
print(board)
```

```
[' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']
```

```
def show_board(board):
    print()
    print(board[0],"|",board[1],"|",board[2])
    print("--+---+--")
    print(board[3],"|",board[4],"|",board[5])
    print("--+---+--")
    print(board[6],"|",board[7],"|",board[8])
    print()
show_board(board)
```

```
| |
--+---+--
| |
--+---+--
| |
```

```
WINNING_LINES=[[0,1,2],[3,4,5],[6,7,8],
                [0,3,6],[1,4,7],[2,5,8],
                [0,4,8],[2,4,6]
                ]
def winner(board):
    for a,b,c in WINNING_LINES:
        if board[a]!=" " and board[a]==board[b]:
            return board[a]
    return None
def empty_boxes(board):
    return[i for i in range(9) if board[i]==" "]
```

```
def minmax(board, is_max_turn, alpha, beta):
    win = winner(board)

    if win == "X":
        return 1
    if win == "O":
        return -1
    if len(empty_boxes(board)) == 0:
        return 0

    if is_max_turn:
        best_score = -2

        for box in empty_boxes(board):
            board[box] = "X"

            score = minmax(board, False, alpha, beta)

            board[box] = " "

            best_score = max(best_score, score)
            alpha = max(alpha, best_score)

            if beta <= alpha:
                break

        return best_score

    else:
        best_score = 2

        for box in empty_boxes(board):
            board[box] = "O"

            score = minmax(board, True, alpha, beta)
```

```

        board[box] = " "

        best_score = min(best_score, score)
        beta = min(beta, best_score)

        if beta <= alpha:
            break

    return best_score

```

```

def best_move(board):
    best_score = -2
    best_box = None

    for box in empty_boxes(board):
        board[box] = "X"
        score = minmax(board, False, -2, 2)
        board[box] = " "

        if score > best_score:
            best_score = score
            best_box = box

    return best_box

```

```

test_board=[
    "X","O","X",
    "O","X"," ",
    " "," ","O",
]
show_board(test_board)
move= best_move(test_board)
print("AI chooses position:",move)

```

```

X | O | X
--+---+--
O | X | 
--+---+--
| | O

```

AI chooses position: 5

```

def play_game():

    board = [" "] * 9

    print("you are o. the ai is x")
    print("choose positions using numbers 0 to 8")

    while True:

        ai_move = best_move(board)
        board[ai_move] = "X"

        print("\nai chooses:", ai_move)
        show_board(board)

        if winner(board) == "X":
            print("ai wins!")
            break

        if len(empty_boxes(board)) == 0:
            print("draw!")
            break

        while True:
            try:
                player_move = int(input("your move(0-8): "))

                if player_move in empty_boxes(board):
                    break

                print("that position is not available")

```

```
except ValueError:
    print("please enter a number from 0 to 8")

board[player_move] = "O"
show_board(board)

if winner(board) == "O":
    print("you win!")
    break

if len(empty_boxes(board)) == 0:
    print("draw!")
    break
```

play_game()

you are o. the ai is x
choose positions using numbers 0 to 8

ai chooses: 0

```
x |  | 
--+-+-
|  | 
--+-+-
|  |
```

your move(0-8): 2

```
x |  | 0
--+-+-
|  | 
--+-+-
|  |
```

ai chooses: 1

```
x | x | 0
--+-+-
|  | 
--+-+-
|  |
```

ai wins!

Start coding or [generate](#) with AI.

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.